

ERROR HANDLING

Scenario 1: Write a stored procedure SafeTransferFunds that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds(
    p_FromAccount NUMBER,
    p_ToAccount NUMBER,
    p_Amount NUMBER
)
AS
    v_Balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_Balance
    FROM Accounts
    WHERE AccountID = p_FromAccount;

    IF v_Balance < p_Amount THEN
        RAISE_APPLICATION_ERROR(-20001,'Insufficient Funds');
    END IF;

    UPDATE Accounts
    SET Balance = Balance - p_Amount
    WHERE AccountID = p_FromAccount;

    UPDATE Accounts
    SET Balance = Balance + p_Amount
    WHERE AccountID = p_ToAccount;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Fund Transfer Successful');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
```

```

        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    END;
/

```

The screenshot displays the FreeSQL SQL Worksheet interface. On the left, the 'Navigator' pane shows a schema named 'My Schema' containing several tables: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main workspace, titled '[SQL Worksheet]*', contains a PL/SQL procedure definition for 'SafeTransferFunds'. The procedure takes three parameters: 'p_FromAccount' (NUMBER), 'p_ToAccount' (NUMBER), and 'p_Amount' (NUMBER). It declares a local variable 'v_Balance' (NUMBER) and begins a block. Inside, it selects the balance from the 'Accounts' table where the account ID matches 'p_FromAccount'. It then checks if the balance is less than the 'p_Amount'. If so, it raises an application error with the message 'Insufficient Funds'. The procedure ends with 'END IF;'. Below the editor, the 'Script output' tab is active, showing the successful execution of the procedure. The output text reads: 'SQL> CREATE OR REPLACE PROCEDURE SafeTransferFunds(...)' followed by 'Show more...', 'Procedure SAFETRANSFERFUNDS compiled', and 'Elapsed: 00:00:00.039'.

```

6      CREATE OR REPLACE PROCEDURE SafeTransferFunds(
7          p_FromAccount NUMBER,
8          p_ToAccount NUMBER,
9          p_Amount NUMBER
10     )
11     AS
12         v_Balance NUMBER;
13     BEGIN
14         SELECT Balance
15         INTO v_Balance
16         FROM Accounts
17         WHERE AccountID = p_FromAccount;
18
19         IF v_Balance < p_Amount THEN
20             RAISE_APPLICATION_ERROR(-20001, 'Insufficient Funds');
21         END IF;
22

```

Query result **Script output** DBMS output Explain Plan SQL history

SQL> CREATE OR REPLACE PROCEDURE SafeTransferFunds(
 p_FromAccount NUMBER,
 p_ToAccount NUMBER,
 p_Amount NUMBER...
[Show more...](#)

Procedure SAFETRANSFERFUNDS compiled

Elapsed: 00:00:00.039

Successful Transfer:

```

BEGIN
    SafeTransferFunds(101,102,500);
END;
/

```

 FreeSQL

>_ Worksheet

 SQL Tutorials

NavigatorFiles

My Schema

Tables

Search





ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*



Aa

1BEGIN

2SafeTransferFunds(101,102,500);

3END;

4/

Query resultScript outputDBMS outputExplain Pl



SQL> BEGIN
SafeTransferFunds(101,102,500);
END;

Fund Transfer Successful

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.028

Error Case:

```
BEGIN
  SafeTransferFunds(101,102,50000);
END;
/
```

The screenshot shows the FreeSQL web interface. The top bar includes the FreeSQL logo, a 'Worksheet' tab, and a 'SQL Tutorials' link. On the left, a 'Navigator' sidebar shows a tree view with 'My Schema' and 'Tables'. Below this is a search bar and a list of database objects: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main area is a SQL editor titled '[SQL Worksheet]*' containing the following code:

```
1 BEGIN
2     ... SafeTransferFunds (101,102,50000);
3 END;
4 /
```

Below the editor, there are tabs for 'Query result', 'Script output', 'DBMS output', and 'Explain Plan'. The 'Script output' tab is active, displaying the following text:

```
SQL> BEGIN
      SafeTransferFunds(101,102,50000);
      END;

Error: ORA-20001: Insufficient Funds

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.161
```

Scenario 2: Write a stored procedure UpdateSalary that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

```
CREATE OR REPLACE PROCEDURE UpdateSalary(
    p_EmployeeID NUMBER,
    p_Percentage NUMBER
)
AS
```

```

BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * p_Percentage / 100)
    WHERE EmployeeID = p_EmployeeID;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20002,'Employee ID does not exist');
    END IF;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully');

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/

```

The screenshot displays the FreeSQL web application interface. On the left, a sidebar contains a 'Navigator' pane with a tree view showing a database schema with tables like ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main area is a 'Worksheet' where a SQL script is entered. The script defines a procedure 'UpdateSalary' that updates the salary of an employee by a given percentage. It includes error handling with a custom error message and a commit statement. The bottom of the interface shows the 'Script output' tab, which displays the execution results, including a confirmation message 'Procedure UPDATESALARY compiled' and the elapsed time '00:00:00.020'.

```

1  CREATE OR REPLACE PROCEDURE UpdateSalary(
2      p_EmployeeID NUMBER,
3      p_Percentage NUMBER
4  )
5  AS
6  BEGIN
7      UPDATE Employees
8      SET Salary = Salary + (Salary * p_Percentage / 100)
9      WHERE EmployeeID = p_EmployeeID;
10
11     IF SQL%ROWCOUNT = 0 THEN
12         RAISE_APPLICATION_ERROR(-20002,'Employee ID does not exist');
13     END IF;
14
15     COMMIT;
16
17     DBMS_OUTPUT.PUT_LINE('Salary Updated Successfully');

```

Query result **Script output** DBMS output Explain Plan SQL history

SQL> CREATE OR REPLACE PROCEDURE UpdateSalary(
 p_EmployeeID NUMBER,
 p_Percentage NUMBER
)...
 Show more...

Procedure UPDATESALARY compiled
 Elapsed: 00:00:00.020

Successful:

```
BEGIN
  UpdateSalary(2,10);
END;
/
```

The screenshot displays an SQL IDE interface. On the left, the 'Navigator' pane shows a tree structure with 'My Schema' and 'Tables'. Below this, a list of tables is visible: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main workspace is titled '[SQL Worksheet]*' and contains the following SQL code:

```
1 BEGIN
2   UpdateSalary(2,10);
3 END;
4 /
```

Below the code editor, the 'Script output' tab is active, showing the execution results:

```
SQL> BEGIN
      UpdateSalary(2,10);
      END;

Salary Updated Successfully

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.011
```

Error:

```
BEGIN
  UpdateSalary(100,10);
END;
/
```

The screenshot displays the FreeSQL web application interface. At the top, there is a navigation bar with the FreeSQL logo, a 'Worksheet' tab, and a link to 'SQL Tutorials'. On the left side, a 'Navigator' panel shows a tree view with 'My Schema' and 'Tables'. Below this, a search bar and a list of database tables (ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, TRANSACTIONS) are visible. The main area is a SQL editor titled '[SQL Worksheet]*' containing the following code:

```
1 BEGIN
2   ...UpdateSalary(100,10);
3 END;
4 /
```

Below the editor, there are tabs for 'Query result', 'Script output', 'DBMS output', and 'Errors'. The 'Script output' tab is currently selected, showing the execution results:

```
SQL> BEGIN
      UpdateSalary(100,10);
      END;

Error: ORA-20002: Employee ID does not exist

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.005
```

Scenario 3: Write a stored procedure AddNewCustomer that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```
CREATE OR REPLACE PROCEDURE AddNewCustomer(
    p_CustomerID NUMBER,
    p_Name VARCHAR2,
    p_DOB DATE,
    p_Balance NUMBER
)
AS
BEGIN
    INSERT INTO Customers(
        CustomerID,
        Name,
        DOB,
        Balance,
        LastModified,
        IsVIP
    )
    VALUES(
        p_CustomerID,
        p_Name,
        p_DOB,
        p_Balance,
        SYSDATE,
        'N'
    );

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Customer Added Successfully');

EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: Customer ID already exists.');
```

```
    WHEN OTHERS THEN
        ROLLBACK;
```



```

        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
    END;
/

```

The screenshot displays the FreeSQL SQL Worksheet interface. On the left, the 'Navigator' pane shows a tree structure with 'My Schema' and 'Tables'. Below this, a list of database objects is visible: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main workspace, titled '[SQL Worksheet]*', contains a SQL script for creating or replacing a procedure named 'AddNewCustomer'. The script defines parameters: p_CustomerID (NUMBER), p_Name (VARCHAR2), p_DOB (DATE), and p_Balance (NUMBER). It begins with an 'INSERT INTO Customers' statement, listing columns: CustomerID, Name, DOB, Balance, LastModified, and IsVIP. The script is partially completed, showing the 'VALUES (' section. Below the script editor, the 'Script output' tab is active, showing the execution results. The output indicates that the procedure 'ADDNEWCUSTOMER' was compiled successfully and provides the elapsed time: 00:00:00.030.

```

1  CREATE OR REPLACE PROCEDURE AddNewCustomer(
2      p_CustomerID NUMBER,
3      p_Name VARCHAR2,
4      p_DOB DATE,
5      p_Balance NUMBER
6  )
7  AS
8  BEGIN
9      INSERT INTO Customers(
10         CustomerID,
11         Name,
12         DOB,
13         Balance,
14         LastModified,
15         IsVIP
16     )
17     VALUES (

```

Query result **Script output** DBMS output Explain Pla

SQL> CREATE OR REPLACE PROCEDURE AddNewCustomer(
 p_CustomerID NUMBER,
 p_Name VARCHAR2,
 p_DOB DATE,...
 Show more...

Procedure ADDNEWCUSTOMER compiled

Elapsed: 00:00:00.030

Successful:

```

BEGIN
    AddNewCustomer(
        6,
        'Sarah Wilson',
        TO_DATE('1998-05-15','YYYY-MM-DD'),
        9000
    );
END;

```

/

The screenshot shows an SQL IDE with a left-hand 'Navigator' pane and a main workspace. The 'Navigator' pane has tabs for 'My Schema' and 'Tables'. Under 'My Schema', there is a search bar and a list of tables: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main workspace is titled '[SQL Worksheet]*' and contains a PL/SQL procedure named 'AddNewCustomer'. The procedure is defined as follows:

```
1 BEGIN
2   AddNewCustomer(
3     6,
4     'Sarah Wilson',
5     TO_DATE('1998-05-15','YYYY-MM-DD'),
6     9000
7   );
8 END;
9 /
```

Below the code editor, there are four tabs: 'Query result', 'Script output', 'DBMS output', and 'Explain Plan'. The 'Script output' tab is selected and shows the following output:

```
6,
'Sarah Wilson',...
Show more...

Customer Added Successfully

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.011
```

Duplicate ID:

```
BEGIN
  AddNewCustomer(
    1,
    'Test User',
    TO_DATE('1995-01-01','YYYY-MM-DD'),
    1000
  );
END;
/
```

NavigatorFiles

My Schema

Tables

Q Search

Q

...

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*

▶

≡

🔗

📄

≡

Aa

▼

1

BEGIN

2

AddNewCustomer (

3

1,

4

'Test User',

5

TO_DATE('1995-01-01','YYYY-MM-DD'),

6

1000

7

);

8

END;

9

/

Query resultScript outputDBMS outputExplain Plan

🗑️

⬇️

AddNewCustomer (

1,

'Test User',...

Show more...

Error: Customer ID already exists.

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.020